

Champion Document — TL1

Name: Chris Bailey **Feature:** UI/HUD, Game State & Menu System **Mark:** ____ / 30

FEATURE REQUIREMENTS BASIS. TL1 owns the game shell: UI screens, HUD, and the overall game-flow State pattern. The contract below keeps dependencies event-driven and testable.

1. Brief Introduction

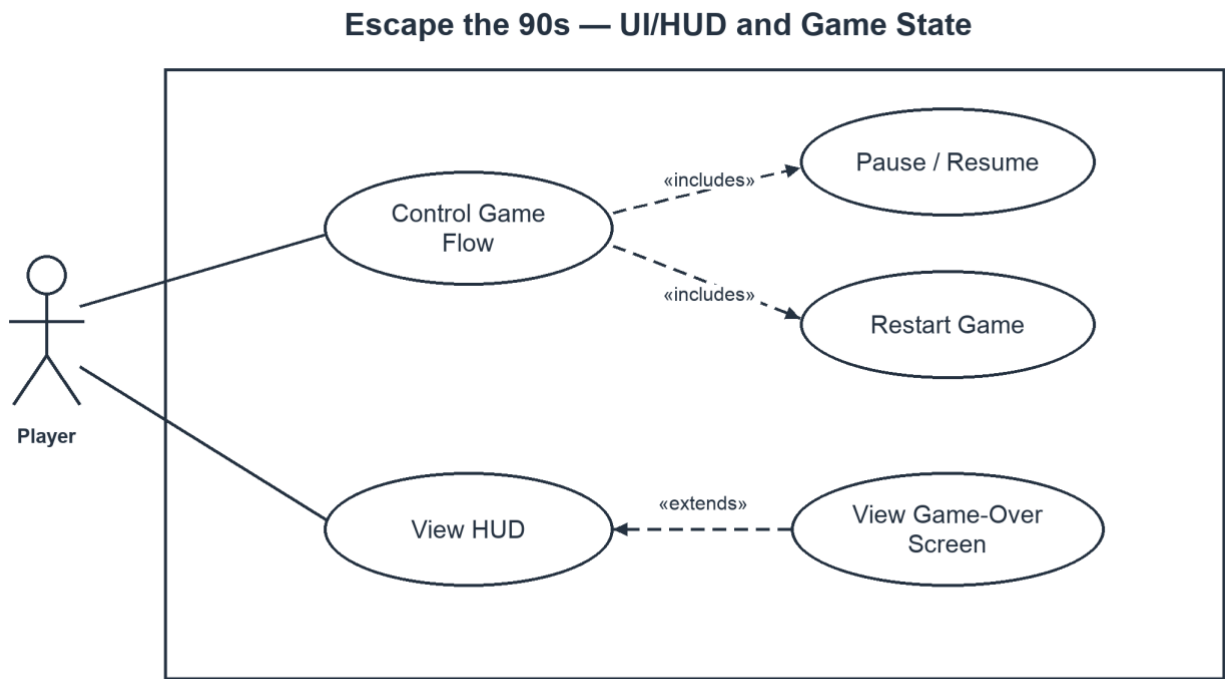
My TL1 feature owns the UI/HUD, the menu system, and the overall game-flow state machine. It is the shell the rest of the game runs inside. GameStateManager tracks the current state and coordinates transitions among MenuState, PlayingState, PausedState, and GameOverState. HUDController displays health, score, and currency and updates those values when typed gameplay events or public update methods are received.

At runtime, the feature handles main-menu entry, start-of-game transition, pause and resume, game-over presentation, and restart. The State pattern provides dynamic binding through virtual GameState.Enter() and GameState.Update() methods; each concrete state overrides those methods so the UI and allowed behavior can differ by state. GameStateManager serves as the GRASP Controller, while HUDController is a Pure Fabrication that keeps display logic separate from state logic and preserves low coupling.

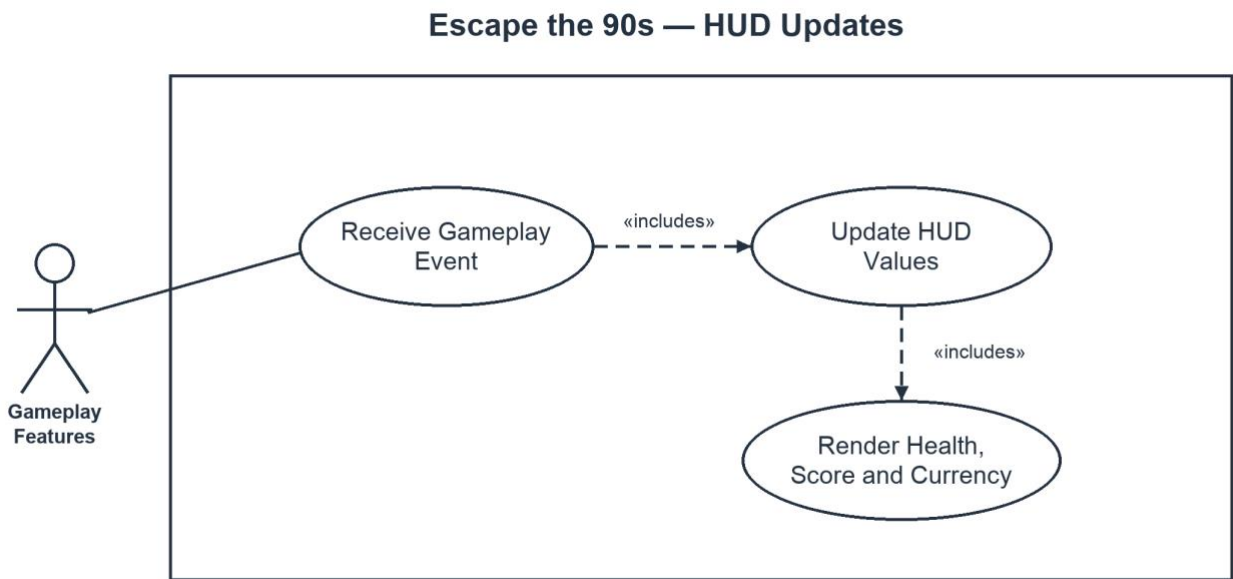
This feature deliberately does not calculate combat damage, decide when enemies are defeated, implement player movement, award power-ups, or own the underlying currency rules. Those behaviors belong to the other gameplay features. TL1 only subscribes read-only to OnPlayerDied, OnPowerUpCollected, OnEnemyDefeated, and currency-update events, then reflects those results in game state or the HUD. It also does not allow external code to set CurrentState directly; state changes must go through the manager requests or subscribed events.

2. Use Case — Diagrams and Scenarios

Use Case Diagrams



Use case diagram 1 — game flow, actor: Player



Use case diagram 2 — HUD updates, actor: Gameplay Features

Scenario 1 — Control Game Flow

Name: Pause, Resume, and Restart

Summary: The player requests a game-flow action; GameStateManager validates the request, changes state, and updates the visible UI.

Actors: Player

Preconditions: GameStateManager is initialized and CurrentState references a valid GameState instance.

Basic sequence:

Step 1: The player selects Start, Pause, Resume, or Restart from an available UI control.

Step 2: The UI calls RequestPause(), RequestResume(), or RequestRestart(), or requests the initial transition from MenuState to PlayingState.

Step 3: GameStateManager checks CurrentState and validates that the requested transition is legal.

Step 4: The current GameState exits and the target state becomes CurrentState.

Step 5: GameStateManager invokes the target state's overridden Enter() behavior.

Step 6: GameStateManager raises OnStateChanged with the new state.

Step 7: UI listeners show/hide the correct HUD, menu, pause overlay, or game-over screen.

Exceptions:

Step 3a: RequestPause() is received while CurrentState is not PlayingState. The manager rejects the request, leaves CurrentState unchanged, and raises no OnStateChanged event.

Step 3b: RequestResume() is received while CurrentState is not PausedState. The request is ignored and the sequence ends without a transition.

Step 4a: RequestRestart() is received from GameOverState. Runtime reset is requested before the manager enters PlayingState; if reset confirmation is unavailable, the manager remains in GameOverState and reports the failed restart to the UI.

Postconditions: CurrentState is valid, exactly one state is active, and visible UI matches that state.

Priority: 1 (must have)

ID: GF1

Scenario 2 — Update HUD from Gameplay Events

Name: Update HUD

Summary: TL1 consumes typed gameplay notifications and updates health, score, or currency without owning the gameplay rules that produced them.

Actors: Gameplay features (Player, Enemy, Power-Up, Currency systems)

Preconditions: HUDController is initialized and subscribed to the required read-only events.

Basic sequence:

Step 1: A gameplay feature publishes a supported event or calls a typed HUD update method.

Step 2: HUDController receives an integer value for the affected HUD field.

Step 3: HUDController validates the value against the field contract.

Step 4: The internal HUD value is updated.

Step 5: The visible health, score, or currency text/bar is refreshed on the next UI update.

Exceptions:

Step 3a: UpdateHealth(int hp) receives $hp < 0$. HUDController clamps the displayed value to 0 and does not display a negative health value.

Step 3b: A duplicate event repeats the same score/health/currency value. HUDController keeps the existing value and does not create an additional state transition.

Step 5a: CurrentState is PausedState, MenuState, or GameOverState and that state hides gameplay HUD elements. The value is retained internally but the gameplay HUD remains hidden until a state that displays it is entered.

Postconditions: The stored HUD value matches the accepted input and the visible HUD is consistent with CurrentState.

Priority: 1 (must have)

ID: HUD1

3. Interface Contract

All externally visible members are typed and have observable pass/fail behavior. External features may read state or publish/trigger supported events, but they do not directly mutate TL1 internal state.

Inputs consumed by this feature

Source	Event / type	Contract	Test / verification
Player Controller (TL6)	OnPlayerDied : event Action	Read-only subscription from the Player feature.	While Playing, the event causes exactly one transition to GameOverState.
Power-Ups & Abilities (TL3)	OnPowerUpCollected : event Action<PowerUpData>	Read-only subscription; TL1 consumes the display-relevant result only.	HUD reflects the supplied display effect or value without modifying power-up ownership.
Enemies & Combat (TL4)	OnEnemyDefeated : event Action<int>	Integer payload is a score delta or agreed score result.	Score display updates exactly once per received event.
Meta Progression (TL5)	CurrencyUpdated : event Action<int>	Integer payload is the authoritative current currency total.	Currency display equals the event payload within 1 UI frame.

Outputs and public interface this feature exposes

Member	Signature / type	Direction and contract	Test / verification
CurrentState	GameStateManager. CurrentState : GameState	Output; read-only active state. External assignment is not exposed.	After any legal transition, value equals the target state before OnStateChanged observers run.
OnStateChanged	GameStateManager. OnStateChanged : event Action<GameState>	Output event; raised once after a successful state change with the new state.	1 event per legal transition; 0 events for rejected or no-op requests.
Pause	GameStateManager. RequestPause() : void	Input; legal only from PlayingState.	Playing → Paused within 1 frame; all other states unchanged.
Resume	GameStateManager. RequestResume() : void	Input; legal only from PausedState.	Paused → Playing within 1 frame; all other states unchanged.
Restart	GameStateManager. RequestRestart() : void	Input; resets runtime and enters PlayingState from GameOverState.	GameOver → Playing after reset; HUD values return to configured start values.

Member	Signature / type	Direction and contract	Test / verification
Update score	HUDController. UpdateScore(int score) : void	Input int; accepts a non-negative score value.	Displayed score equals input within 1 UI frame; negative input is rejected or clamped per implementation test.
Update health	HUDController. UpdateHealth(int hp) : void	Input int; values below 0 display as 0.	Displayed health equals max(0, hp) within 1 UI frame.

State classes and dynamic binding

abstract GameState defines virtual Enter() and Update(). MenuState, PlayingState, PausedState, and GameOverState override those methods. A regression test must verify that entering PausedState hides or disables gameplay presentation that remains active in PlayingState; this specifically demonstrates that the overrides are being dispatched dynamically rather than all states behaving identically.

Process Description — Context Diagram

Context Diagram: Escape the 90s

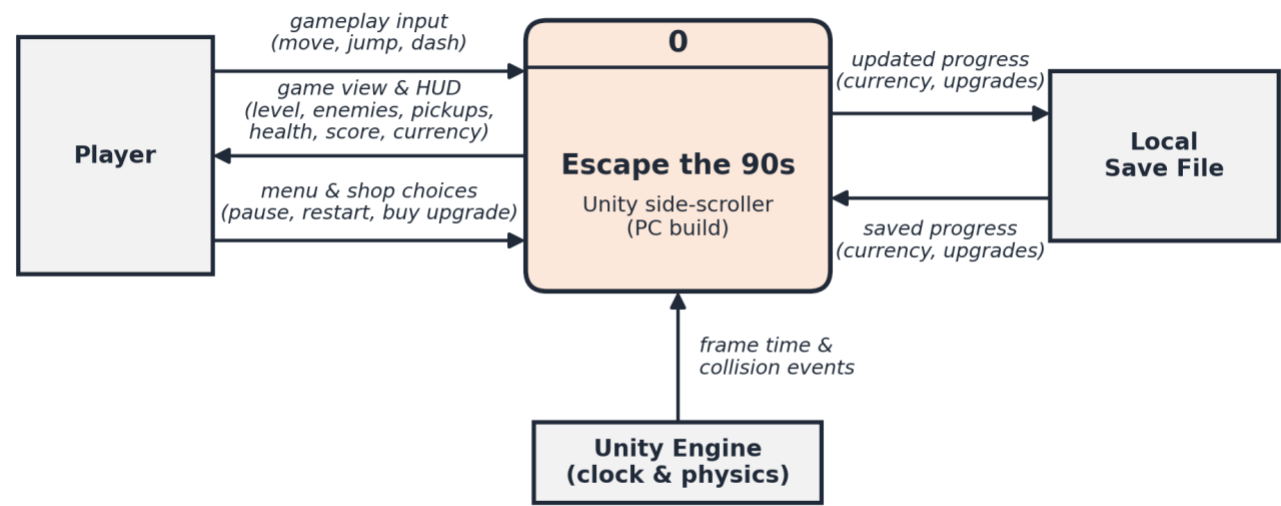


Diagram 0 decomposes Process 0 into the six features: TL1 UI/HUD & Game State, TL2 Level Generation, TL3 Power-Ups, TL4 Enemies & Combat, TL5 Meta Progression, TL6 Player Controller. Every flow above must reappear there.

Context diagram — Escape the 90s whole-game scope

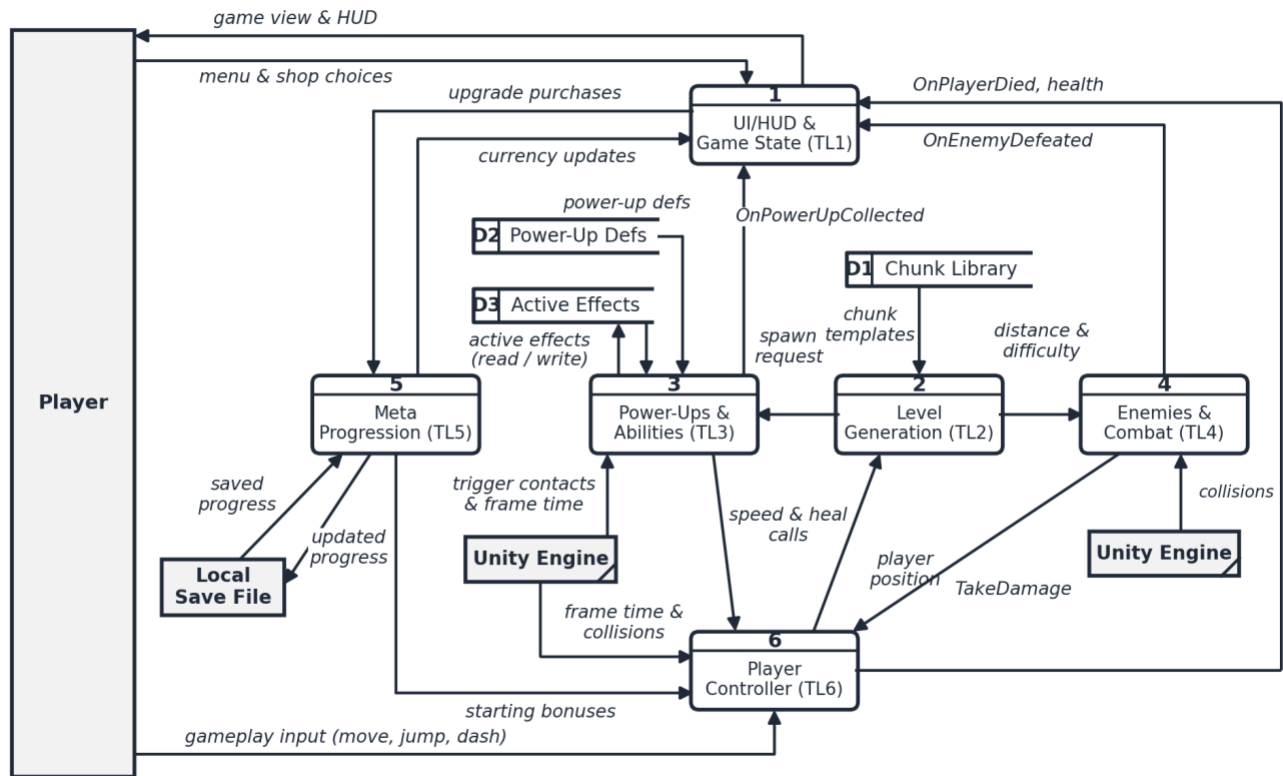
This context diagram shows Escape the 90s as Process 0 and its external flows with the Player, Local Save File, and Unity Engine. The team-supplied Diagram 0 below decomposes the whole game into TL1 through TL6. These two diagrams establish project scope; Diagram 1 is the detailed decomposition of my TL1 UI/HUD, Game State & Menu System feature.

4. Data Flow Diagrams — Gane–Sarson

Diagram 1 expands only TL1. Player menu & shop choices enter TL1; TL1 returns the game view & HUD and forwards upgrade purchases to TL5. TL1 receives OnPlayerDied and health from TL6, OnEnemyDefeated from TL4, OnPowerUpCollected from TL3, and currency updates from TL5. Its internal processes route these inputs through

menu input, game-state management, and HUD updating. Each process produces a data or UI result from its inputs, avoiding black holes and miracles. TL1 does not calculate the gameplay results owned by other features.

Diagram 0: Escape the 90s



Unity Engine appears twice (duplicate entity, marked by the corner stroke). Process numbers match TL numbers.

Diagram 0 — team-wide Escape the 90s decomposition

Diagram 1: TL1 UI/HUD and Game State zoom

TL1-owned processes in blue; external features in gray

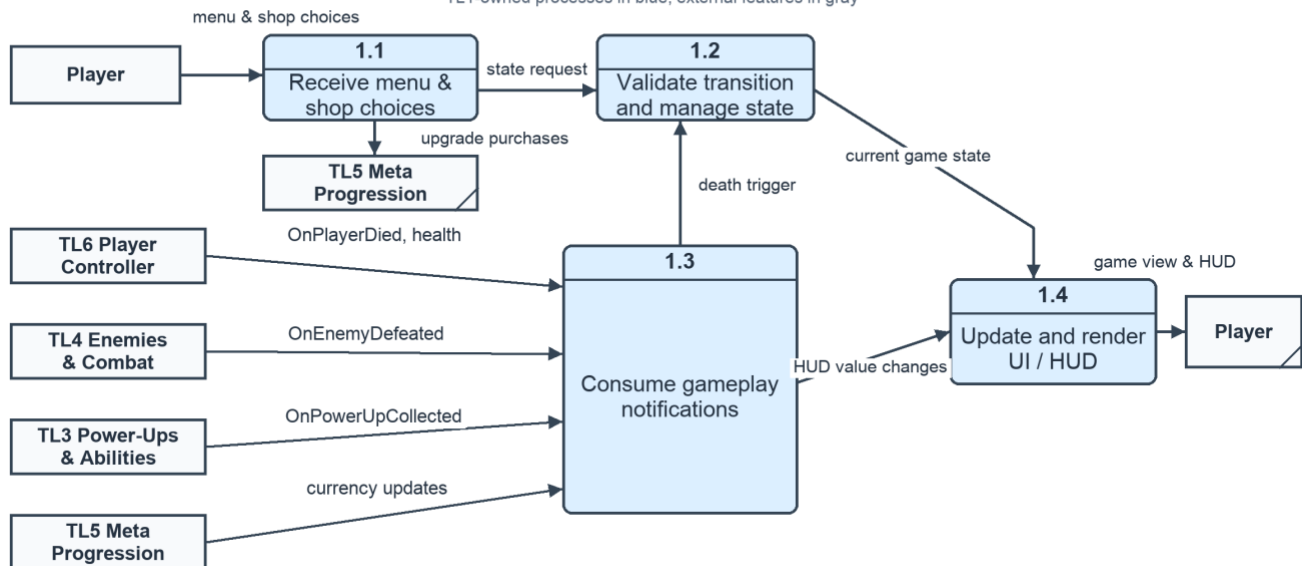


Diagram 1 — TL1 UI/HUD, Game State and Menu System zoom

Primitive process description — 1.2 Validate Transition

The transition validator receives the requested action plus CurrentState. It emits either a valid target GameState or a no-op/rejection. Legal transitions used by the core feature are Menu → Playing, Playing → Paused, Paused → Playing, Playing → GameOver when OnPlayerDied fires, and GameOver → Playing after restart/reset. This keeps transition policy inside GameStateManager instead of scattering it across UI buttons.

5. Timeline

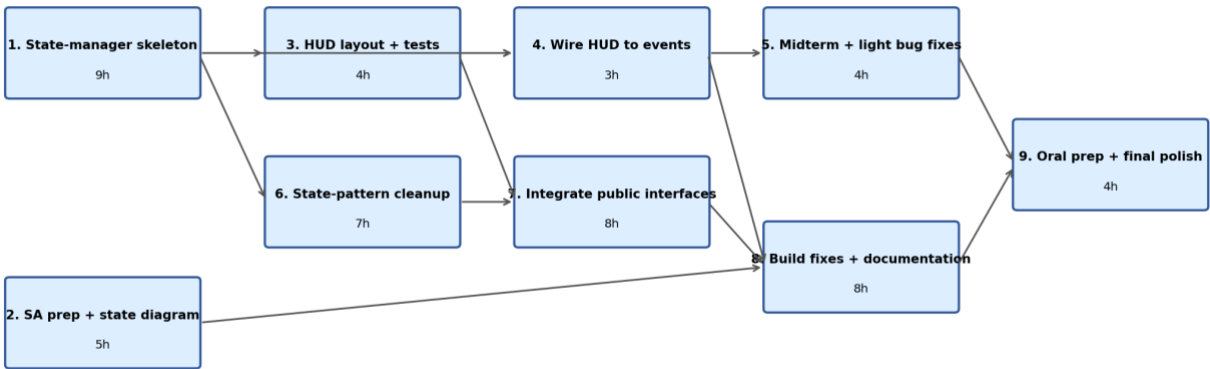
The supplied TL1 effort is 52 hours: Design 9h, Coding 24h, Testing 10h, and Documentation 9h. The work items below preserve those totals rather than shrinking the estimate to fit an optimistic implementation path.

Task	Category	Duration (hours)	Predecessor task(s)
1. State-manager skeleton	Parent task	9	—
1.1 State/interface design	Design	4	—
1.2 GameState base + manager	Coding	5	1.1
2. SA prep + state diagram	Parent task	5	—
2.1 Use-case + UI flow design	Design	5	—
3. HUD layout + tests	Parent task	4	1
3.1 HUDController + bindings	Coding	4	1.1
4. Wire HUD to events	Parent task	3	1, 3
4.1 Integration/event wiring	Testing	3	1.2, 3.1
5. Midterm + light bug fixes	Parent task	4	4
5.1 Boundary + rapid-toggle stress tests	Testing	4	4.1
6. State-pattern cleanup	Parent task	7	1
6.1 Concrete state classes	Coding	7	1.2
7. Integrate public interfaces	Parent task	8	3, 6
7.1 Pause/resume/restart flow	Coding	5	6.1
7.2 Menus + game-over UI wiring	Coding	3	3.1, 6.1
8. Build fixes + documentation	Parent task	8	2, 4, 7
8.1 Unit transition tests	Testing	3	4.1, 7.1, 7.2
8.2 Champion documentation	Documentation	5	2.1, 4.1
9. Oral prep + final polish	Parent task	4	5, 8
9.1 Diagrams + final documentation QA	Documentation	4	5.1, 8.1, 8.2

Category totals: Design 9h · Coding 24h · Testing 10h · Documentation 9h = 52h.

PERT diagram

PERT — TL1 Parent Work Items

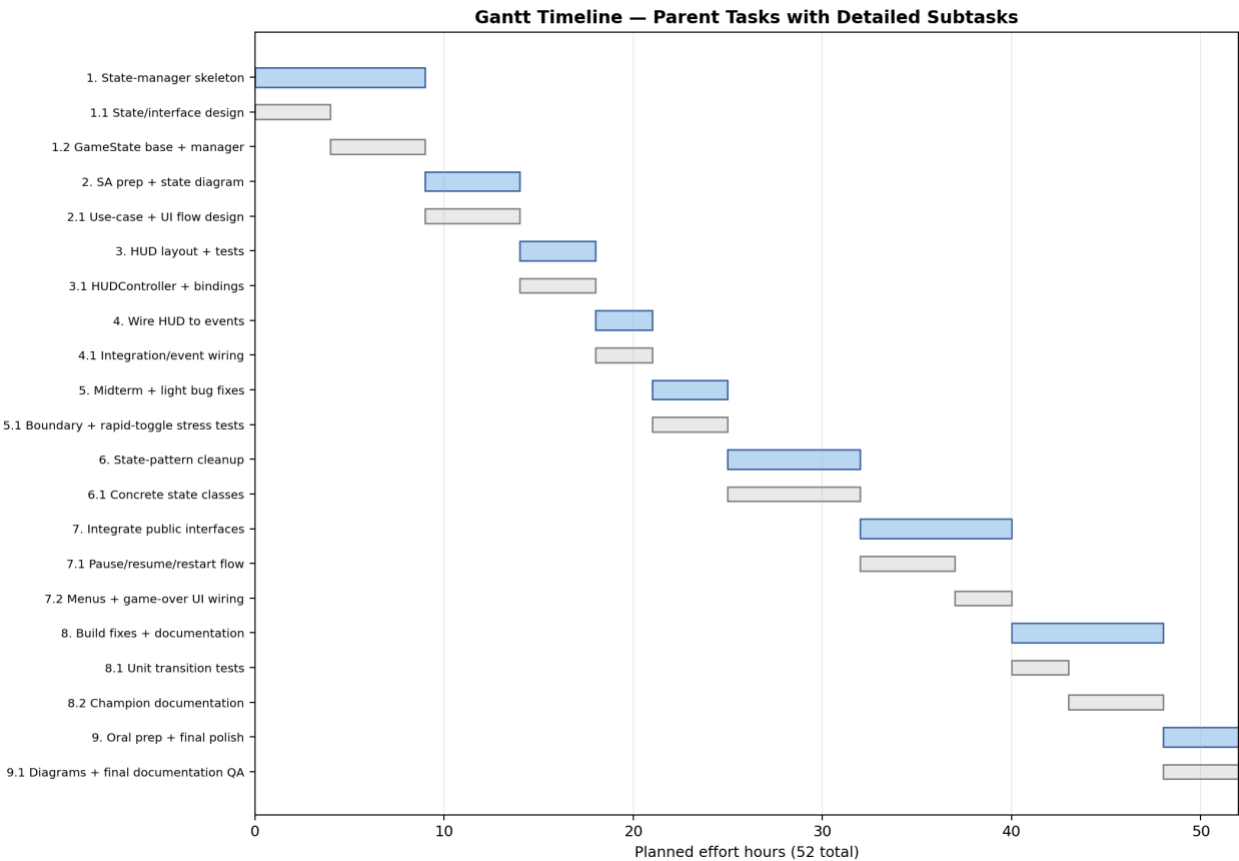


Parent durations equal the sum of their detailed subtasks. Total planned effort: 52 hours.

PERT diagram — TL1 parent task dependencies

The PERT view uses the nine schedule items as parent tasks. Each parent duration is the sum of its detailed subtasks from the original 52-hour estimate; the arrows show the major parent-level dependencies while the table preserves the more specific predecessor relationships.

Gantt timeline



Gantt timeline — TL1 parent tasks and detailed subtasks

TL4 tracking checkpoints

Track actual hours against the four supplied effort buckets. A variance greater than 2 hours in any bucket should be explained in TL4 with the cause (scope change, integration issue, test failure, or underestimated documentation) and the revised remaining estimate. Completion means: HUD values update in real time; pause/resume/restart function from their legal states; game-over displays on player death; restart returns to a clean PlayingState; and rapid pause/resume does not leave duplicate overlays, multiple active states, or repeated OnStateChanged events.